

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра «Програмна інженерія»
Звіт з лабораторної роботи №1
З дисципліни "Генеративні нейронні мережі та їх застосування"
Використання та fine-tuning великих мовних моделей

Виконала:
студентка гр. ПЗМ-25-2
Тимощенко Владислава Русланівна

Перевірила: доцент кафедри ПІ
Шевченко Олена Леонідівна

Харків 2026

1 Мета: вміти розгортати та використовувати попередньо навчені LLM, проводити їх донавчання (fine-tuning) для виконання специфічних задач за допомогою бібліотеки LoRA та оцінювати якість отриманої донавченої моделі.

2 Хід роботи

2.1 Видалимо з шаблону ноутбуку навчання та тестування роботи моделі на наборі даних leprechaun та залишимо лише роботу з yoda стилем. Для цього видалимо всі блоки з (style= «leprechaun»)

```
]: train_loader, test_loader = mdl.lab3.create_dataloader(style="leprechaun")

sample = train_loader.dataset[44]
question = sample['instruction']
answer = sample['response']
answer_style = sample['response_style']

print(f"Question: {question}\n\n" +
      f"Original Answer: {answer}\n\n" +
      f"Answer Style: {answer_style}")
```

2.2 Для відтворюваності результатів задамо на початку ноутбуку (після імпортів) початковий seed наступним чином:

```
import os
import json
import random
import numpy as np
from tqdm import tqdm
import matplotlib.pyplot as plt

import torch
from torch.nn import functional as F
from torch.utils.data import DataLoader

from transformers import AutoTokenizer, AutoModelForCausalLM
from datasets import load_dataset
from peft import LoraConfig, get_peft_model
from lion_pytorch import Lion
from itertools import islice

from transformers import set_seed
SEED = 349058
set_seed(SEED)
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

Про всяк випадок повторимо `set_seed(SEED)` перед створенням `dataloader` та перед `evaluation`, щоб не боятися перезапустити окремі клітинки індивідуально (а не з повним ноутбуком)

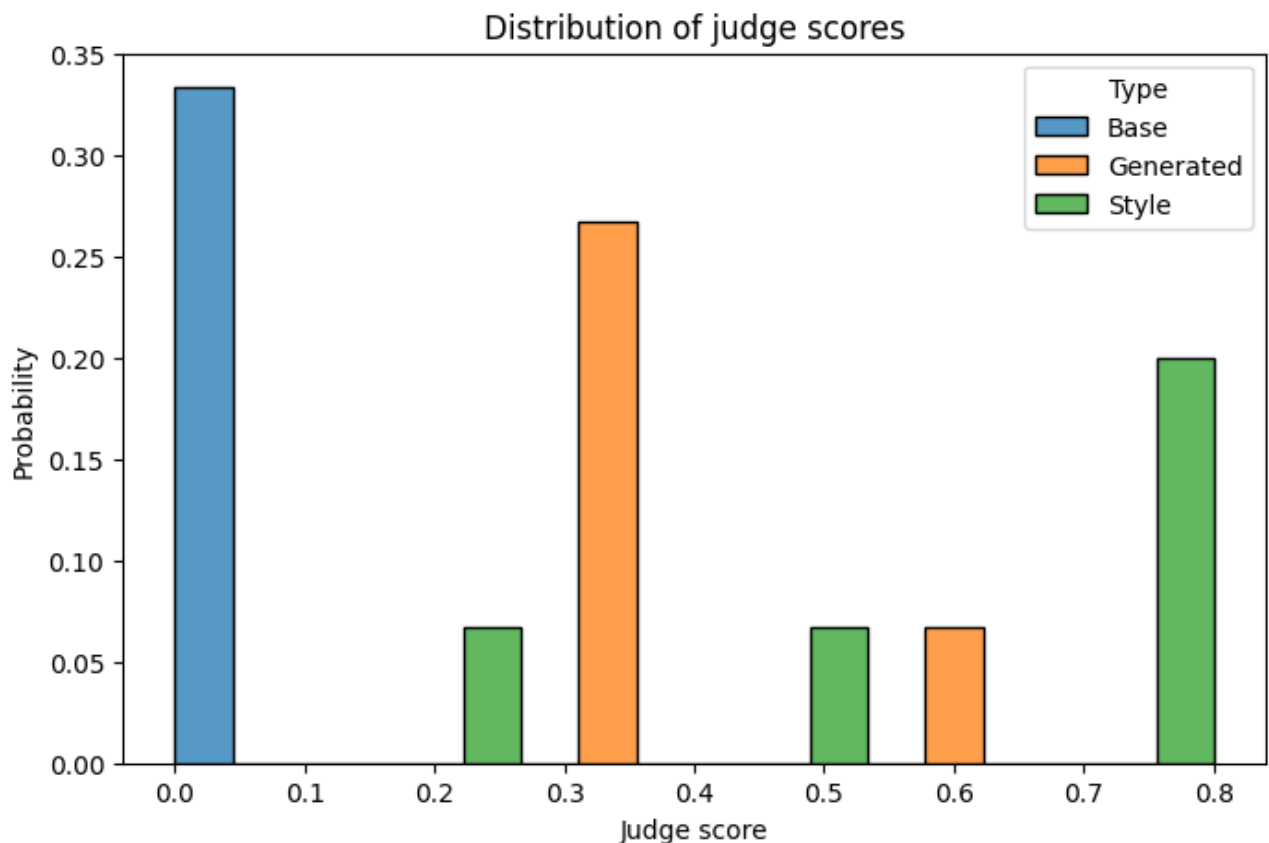
```
# Load the Yoda-speak dataset and fine-tune the model using your training function
set_seed(SEED)
train_loader, test_loader = mdl.lab3.create_dataloader(style="yoda")
model = train(model, train_loader, tokenizer, max_steps=1600, learning_rate=5e-5, run_name="yoda")
```

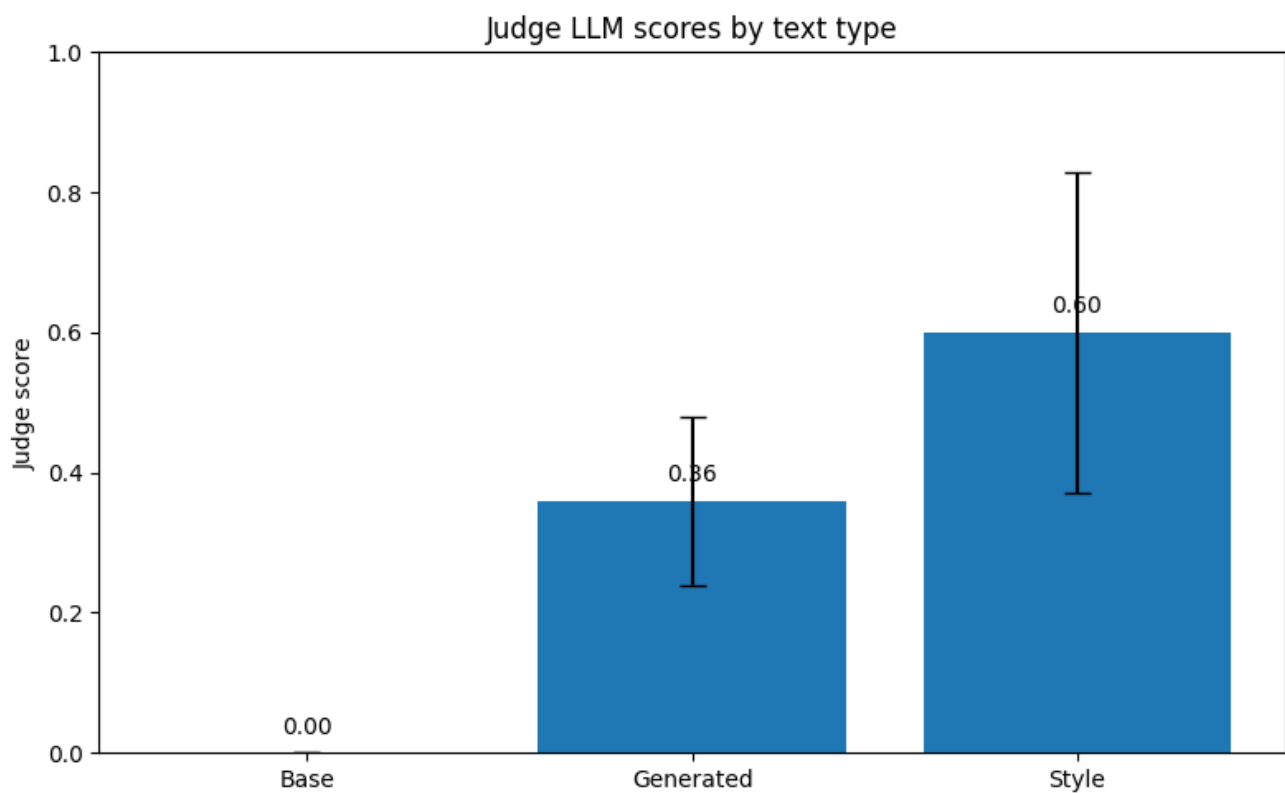
```
set_seed(SEED)
judge = LLMJudgeEvaluator(llm, system_prompt=system_prompt)
```

2.3 Експеримент та його результати

Проведемо 2-3 повтори всього експерименту з різними `seed`. Усереднимо оцінку якості моделі на основі цих результатів. Для цього запусимо всі блоки, спершу із сідом `349058`.

Тепер, коли ми маємо `samples`, можемо оцінити їх за допомогою моделі LLM-судді.





IMPORTANT: RUN THE FOLLOWING CELL BELOW TO PRINT THE RESULT BUT DO NOT MODIFY ITS CONTENTS.

```
# DO NOT CHANGE/MODIFY THIS CELL.
# EXECUTE IT BEFORE SUBMITTING YOUR ENTRY TO THE LAB.

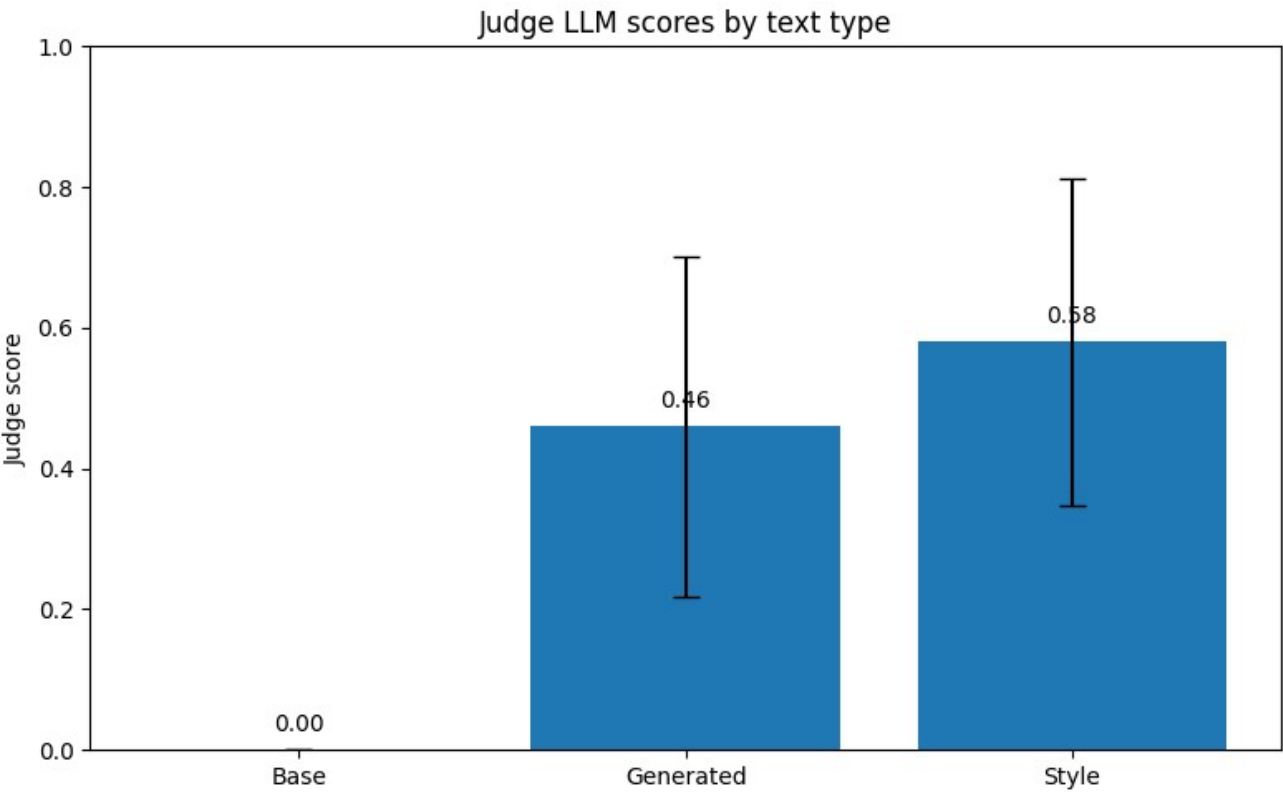
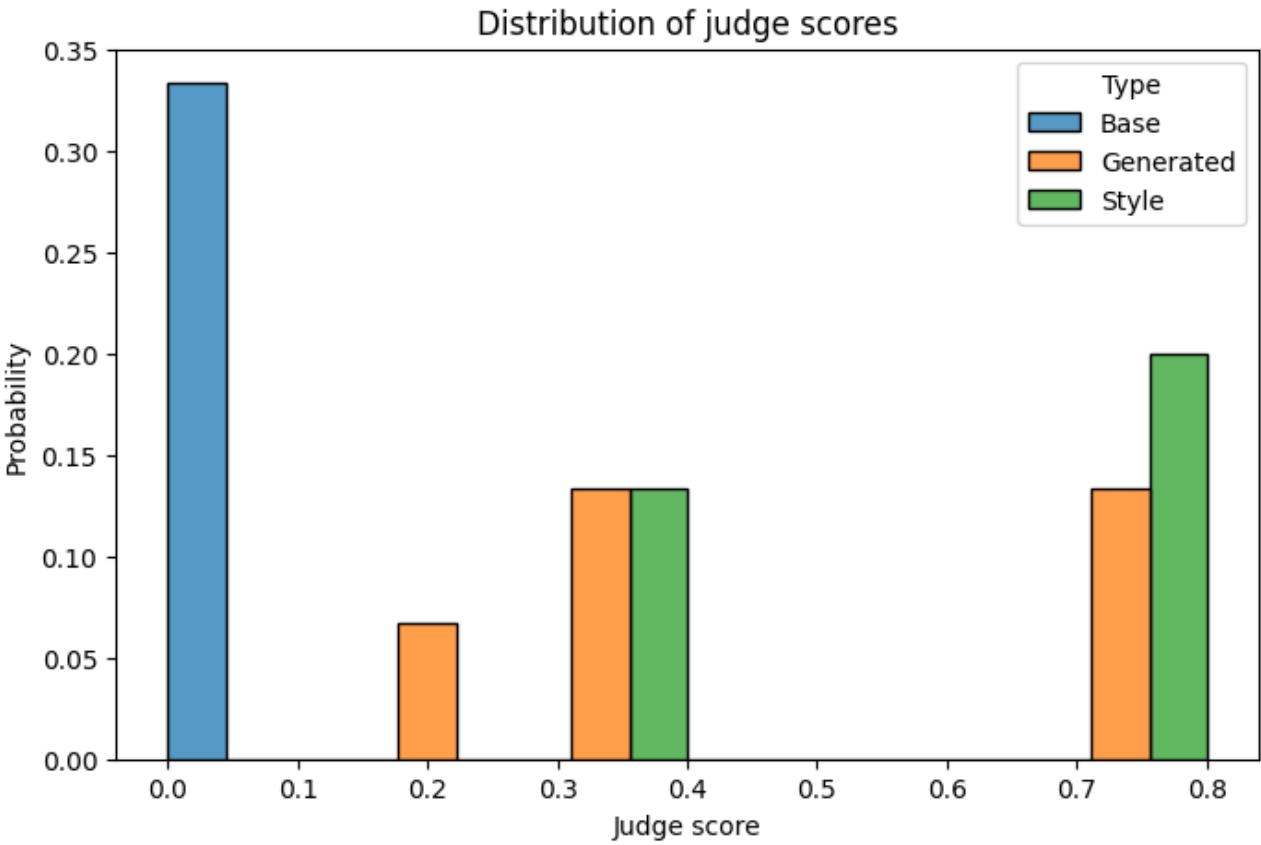
yoda_test_text = mdl.lab3.yoda_test_text
tokens = tokenizer(yoda_test_text, return_tensors="pt").to(model.device)

# Get the loglikelihood from the model
with torch.no_grad():
    outputs = model(**tokens)
    logits = outputs.logits[:, :-1]
    targets = tokens.input_ids[:, 1:]
    loss = F.cross_entropy(logits.reshape(-1, logits.size(-1)),
                           targets.reshape(-1))

print(f"Yoda test loglikelihood: {loss.item():.2f}")
```

... Yoda test loglikelihood: 3.05

Тепер запусимо всі блоки із сідом 983457



```
[50]
✓ Os ● # DO NOT CHANGE/MODIFY THIS CELL.
      # EXECUTE IT BEFORE SUBMITTING YOUR ENTRY TO THE LAB.

      yoda_test_text = mdl.lab3.yoda_test_text
      tokens = tokenizer(yoda_test_text, return_tensors="pt").to(model.device)

      # Get the loglikelihood from the model
      with torch.no_grad():
          outputs = model(**tokens)
          logits = outputs.logits[:, :-1]
          targets = tokens.input_ids[:, 1:]
          loss = F.cross_entropy(logits.reshape(-1, logits.size(-1)),
                                targets.reshape(-1))

      print(f"Yoda test loglikelihood: {loss.item():.2f}")

... Yoda test loglikelihood: 3.08
```

2.4 Аналіз Експерименту

Отже, за результатами першого запуску значення LogLikelihood становило 3,05, а за результатами другого — 3,08. Середнє значення метрики склало 3,065, що свідчить про стабільність роботи моделі та незначне відхилення результатів між окремими запусками.

Оцінювання Judge LLM виконувалося для трьох типів текстів: Base, Generated та Style. Контрольні тексти типу Base в обох запусках отримали оцінку 0,00. Для текстів типу Generated було отримано оцінки 0,36 та 0,46, що відповідає середньому значенню 0,41. Для текстів типу Style отримано оцінки 0,60 та 0,58 із середнім значенням 0,59.

Отримані результати свідчать про те, що тексти після стилізації оцінюються значно вище, ніж початково згенеровані тексти. Це підтверджує ефективність застосованого підходу до стилізації та його позитивний вплив на якість кінцевого результату.

На гістограмах оцінок Judge Score для обох запусків аналіз розподілу показує, що оцінки для текстів типу Style переважно зосереджені у верхній частині шкали (0,5–0,8), тоді як оцінки для Generated мають ширший розкид і

частіше знаходяться в середньому діапазоні. Подібність розподілів у двох незалежних запусках підтверджує відтворюваність отриманих результатів та стабільність роботи моделі.

Таким чином, за результатами проведеного дослідження модель продемонструвала стабільні показники якості. Середнє значення LogLikelihood становить 3,065, середня оцінка Generated-текстів — 0,41, а Style-текстів — 0,59. Це свідчить про успішне відтворення цільового стилю та загалом задовільну якість генерації тексту.

Далі виконаємо завдання на оцінку «добре»:

2.5 Додавання метрики якості навчання

Додамо метрику StyleGain для кількісної оцінки приросту якості стилю після fine-tuning. Метрика визначає різницю між середніми оцінками судді для текстів, згенерованих моделлю після навчання, та базовими текстами до навчання.

Код метрики:

```
def compute_stylegain(base_scores, generated_scores):  
    """  
    StyleGain = mean(generated) - mean(base)  
    """  
    stylegain = np.mean(generated_scores) - np.mean(base_scores)  
    return stylegain * 100  
  
stylegain = compute_stylegain(base_scores, generated_scores)  
  
print(f"=== StyleGain Metric ===")  
print(f"StyleGain: {stylegain:.1f}%")
```

Функція compute_stylegain обчислює середнє значення оцінок LLM-judge для обох груп текстів. Середня оцінка базової моделі (generated_scores) віднімається від середньої оцінки fine-tuned моделі. Різниця множиться на 100 для представлення результату у відсотках на шкалі 0-100.

Результати виконання:

```
=== StyleGain Metric ===  
StyleGain: 46.0%
```

Значення StyleGain становить 46.0%. Приріст на 46 пунктів указує на ефективну адаптацію моделі до генерування текстів у стилі Yoda.

2.6 Перемішування та розділення вибірки

Модифікуємо функцію створення навчальної вибірки, а саме *create_dataloader*. Початкова реалізація використовувала лише навчальну та тестову вибірки, а також розмір батчу `batch_size = 1`. У ході роботи дані були випадковим чином перемішані та розділені на три підмножини: навчальну (`train`), валідаційну (`validation`) та тестову (`test`) у співвідношенні 70/15/15.

Для цього було створено власну реалізацію функції `create_dataloader_split()`, яка виконує:

- завантаження набору даних `databricks/databricks-dolly-15k`;
- завантаження стилізованих відповідей з файлу `yoda.txt`;
- випадкове перемішування даних;
- розділення вибірки на `train`, `validation` та `test`;
- створення `DataLoader` із заданим розміром батчу.

Адаптована функція:

your chat model follow Yoda speak, and then use that information to improve the model.

```
[19]
✓ Os
from datasets import load_dataset
from torch.utils.data import DataLoader
from sklearn.model_selection import train_test_split

[20]
✓ Os
def create_dataloader_split(
    style,
    batch_size=16,
    train_ratio=0.70,
    val_ratio=0.15,
    test_ratio=0.15,
    seed=SEED,
):
    assert abs(train_ratio + val_ratio + test_ratio - 1.0) < 1e-6

    # Завантаження Dolly
    ds = load_dataset(
        "databricks/databricks-dolly-15k",
        split="train"
    )

    # Завантаження стилізованих відповідей
    style_path = os.path.join(
        mdl.lab3.cwd,
        "data",
        "text_styles",
        f"{style}.txt"
    )

    with open(style_path, "r") as f:
        new_responses = [
            line.strip().replace("\\n", "\n")
            for line in f
        ]

    # Формування стилізованого датасету
    ds_style = ds.select(range(len(new_responses)))

    ds_style = ds_style.map(
        lambda x, idx: {"response_style": new_responses[idx]},
        with_indices=True,
        num_proc=1,
    )
```

```
# Перемішування
ds_style = ds_style.shuffle(seed=seed)

# ----- Split -----

first_split = ds_style.train_test_split(
    test_size=test_ratio,
    seed=seed,
)

train_val_ds = first_split["train"]
test_ds = first_split["test"]

val_fraction = val_ratio / (train_ratio + val_ratio)

second_split = train_val_ds.train_test_split(
    test_size=val_fraction,
    seed=seed,
)

train_ds = second_split["train"]
val_ds = second_split["test"]

print(
    f"Train: {len(train_ds)}, "
    f"Validation: {len(val_ds)}, "
    f"Test: {len(test_ds)}"
)

train_loader = DataLoader(
    train_ds,
    batch_size=batch_size,
    shuffle=True,
)

val_loader = DataLoader(
    val_ds,
    batch_size=batch_size,
)

test_loader = DataLoader(
    test_ds,
    batch_size=batch_size,
)

return train_loader, val_loader, test_loader
```

```
[6]
✓ m
# Load the Yoda-speak dataset and fine-tune the model using your training function
set_seed(SEED)
train_loader, val_loader, test_loader = create_dataloader_split(
    style="yoda",
    batch_size=5
)

model = train(model, train_loader, tokenizer, max_steps=1600, learning_rate=5e-5, run_name="yoda")
```

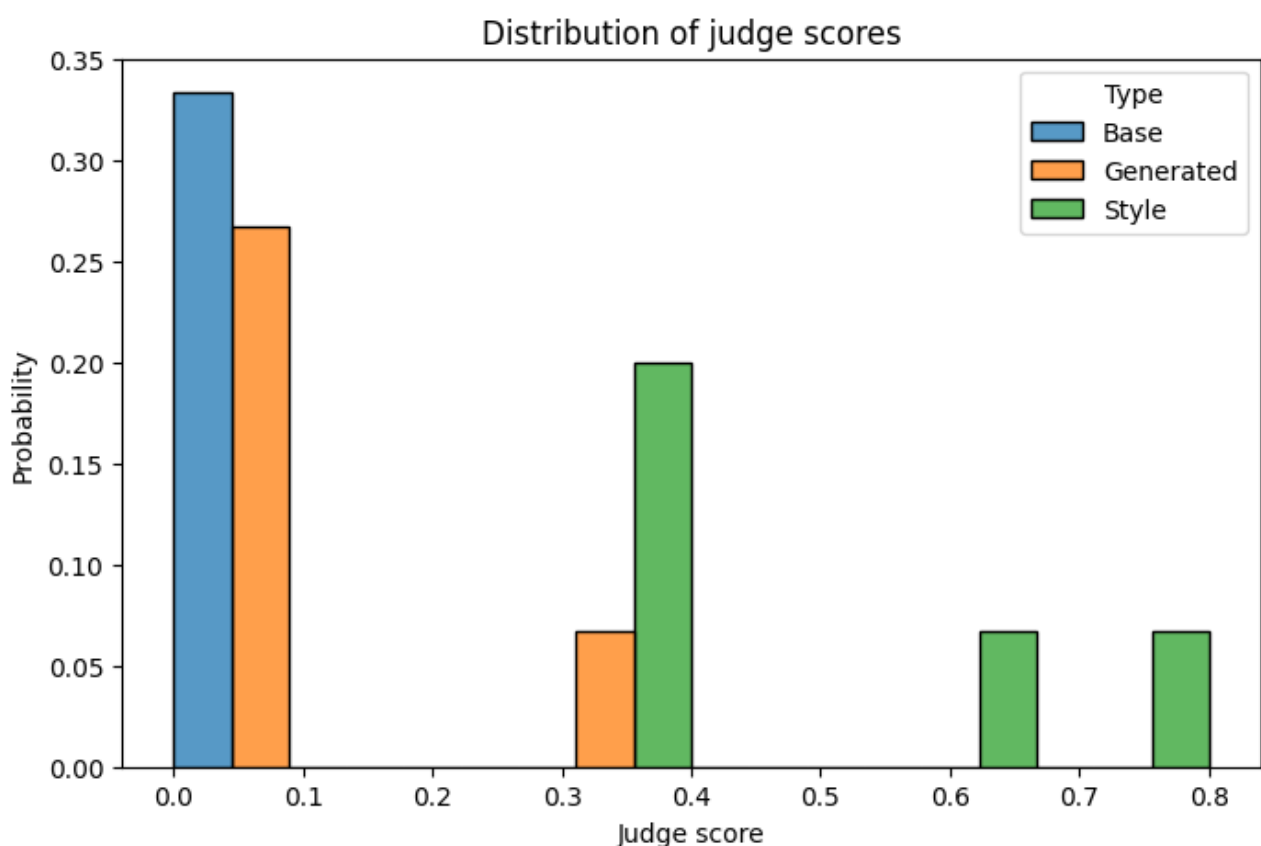
Крім того, розмір батчу було збільшено до `batch_size = 16`, що дозволило скоротити кількість кроків навчання та прискорити процес тренування моделі.

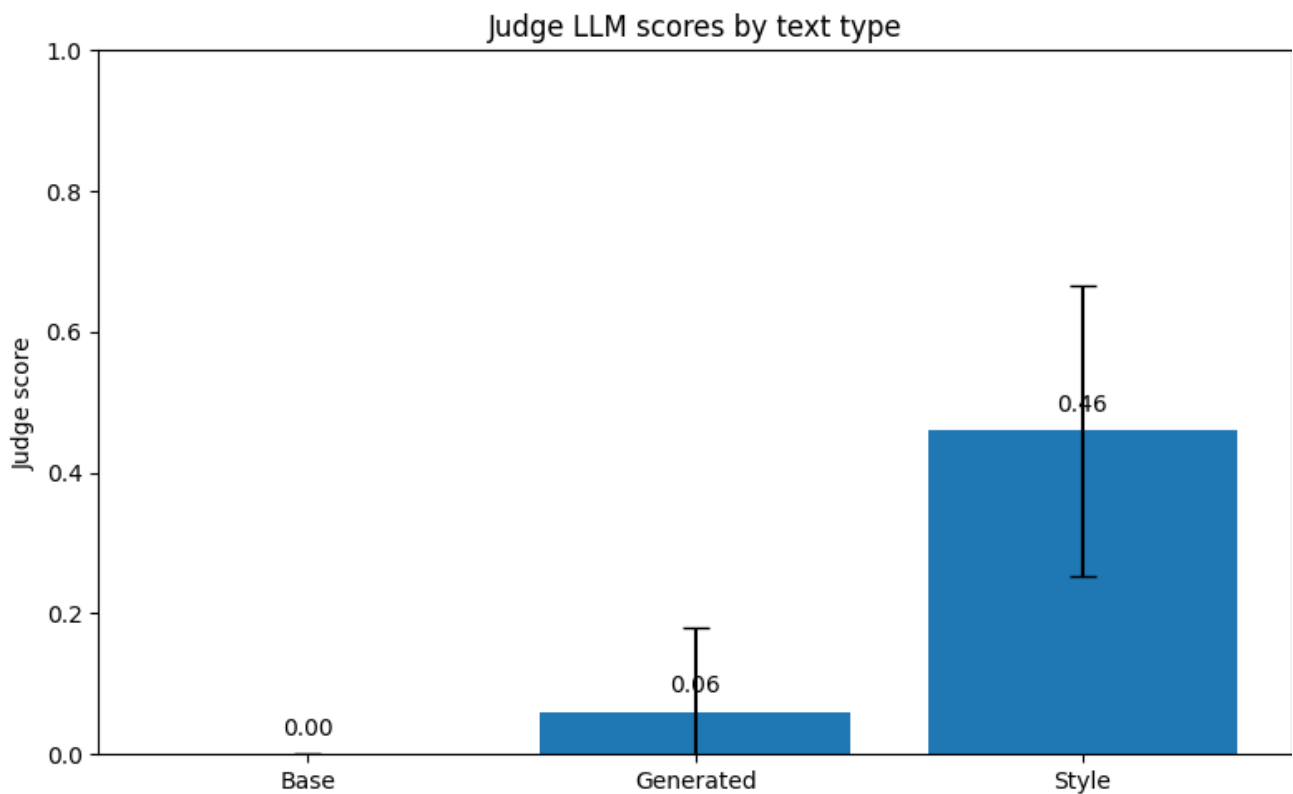
Після розділення даних отримані такі розміри вибірок:

- Train: 1432 приклади;
- Validation: 308 прикладів;
- Test: 308 прикладів.

Отже, основні зміни у коді полягали в додаванні етапу перемішування даних, реалізації розділення на три підвибірки та зміні параметра `batch_size` з 1 до 16.

2.7 Результати після змін та їх аналіз





```
=== StyleGain Metric ===  
StyleGain: 6.0%
```

```
Yoda test loglikelihood: 3.50
```

ПЗ результатів видно, що після розділення вибірки на train/validation/test у співвідношенні 70/15/15 та збільшення розміру батчу до 16 було отримано значення LogLikelihood = 3,50. Оцінювання Judge LLM показало середні оцінки 0,06 для текстів типу Generated та 0,46 для текстів типу Style. Значення метрики StyleGain становило 6,0 %.

Порівняно з попереднім експериментом спостерігається зниження оцінок Judge LLM: для Generated-текстів з 0,41 до 0,06, а для Style-текстів з 0,59 до 0,46. Також зменшилося значення StyleGain р 46% до 6%, що свідчить про менший ефект стилізації.

Отримані результати показують, що після зміни способу підготовки даних і збільшення розміру батчу до 16 якість відтворення стилю Йоди дещо погіршилася. Середні оцінки Judge LLM та значення StyleGain виявилися нижчими порівняно з попереднім експериментом. Водночас використання більшого розміру батчу дозволило суттєво скоротити кількість кроків навчання та прискорити процес тренування моделі. Таким чином, було досягнуто виграшу в швидкості навчання ціною певного зниження якості стилізації тексту.

2.8 Дослідження впливу параметра LoRA rank на якість моделі

Для оцінки впливу параметру LoRA на якість стилізації моделі проведемо серію експериментів. В усіх запусках змінюватимемо лише параметр LoRA rank (r), а значення learning rate та context length не чіпатимемо.

Програмний код:

```
[74]
✓ 16m
import pandas as pd
from transformers import AutoModelForCausalLM

# =====
# ПАРАМЕТР ДЛЯ ДОСЛІДЖЕННЯ
# =====

LORA_RANKS = [4, 8, 16, 32]

# фіксовані параметри
LEARNING_RATE = 5e-5
CONTEXT_LENGTH = 512
MAX_STEPS = 1600

experiment_results = []

for rank in LORA_RANKS:

    print("=" * 60)
    print(f"LoRA rank = {rank}")
    print("=" * 60)

    # Завантажуємо базову модель заново
    model = AutoModelForCausalLM.from_pretrained(
        model_id,
        device_map="auto"
    )

    # Застосовуємо LoRA
    model = apply_lora(model, rank)

    # Кількість тренованих параметрів
    trainable_params = sum(
        p.numel() for p in model.parameters()
        if p.requires_grad
    )

    # Навчання
    model = train(
        model,
        train_loader,
        tokenizer,
        max_steps=MAX_STEPS,
```

```

[74] 16m
learning_rate=LEARNING_RATE,
context_length=CONTEXT_LENGTH,
run_name=f"rank_{rank}"
)

# Генерація
generated_samples = generate_samples_from_test(
    test_loader,
    num_samples=n_samples
)

# Оцінювання
generated_scores = evaluate_samples(
    generated_samples,
    f"generated_rank_{rank}"
)

base_scores = evaluate_samples(
    base_samples,
    f"base_rank_{rank}"
)

stylegain = compute_stylegain(
    base_scores,
    generated_scores
)

experiment_results.append({
    "LoRA_rank": rank,
    "Trainable_params": trainable_params,
    "Generated_score_mean": np.mean(generated_scores),
    "Generated_score_std": np.std(generated_scores),
    "StyleGain": stylegain
})

# =====
# ПІДСУМКОВА ТАБЛИЦЯ
# =====

results_df = pd.DataFrame(experiment_results)

display(results_df)

```

```

[74] 16m
# =====
# ПІДСУМКОВА ТАБЛИЦЯ
# =====

results_df = pd.DataFrame(experiment_results)

display(results_df)

# =====
# ГРАФІК
# =====

plt.figure(figsize=(8,5))

plt.plot(
    results_df["LoRA_rank"],
    results_df["Generated_score_mean"],
    marker="o"
)

plt.xlabel("LoRA Rank")
plt.ylabel("Mean Judge Score")
plt.title("Effect of LoRA Rank on Model Quality")
plt.grid(True)

plt.show()

# =====
# НАЙКРАЩА КОНФІГУРАЦІЯ
# =====

best_run = results_df.loc[
    results_df["Generated_score_mean"].idxmax()
]

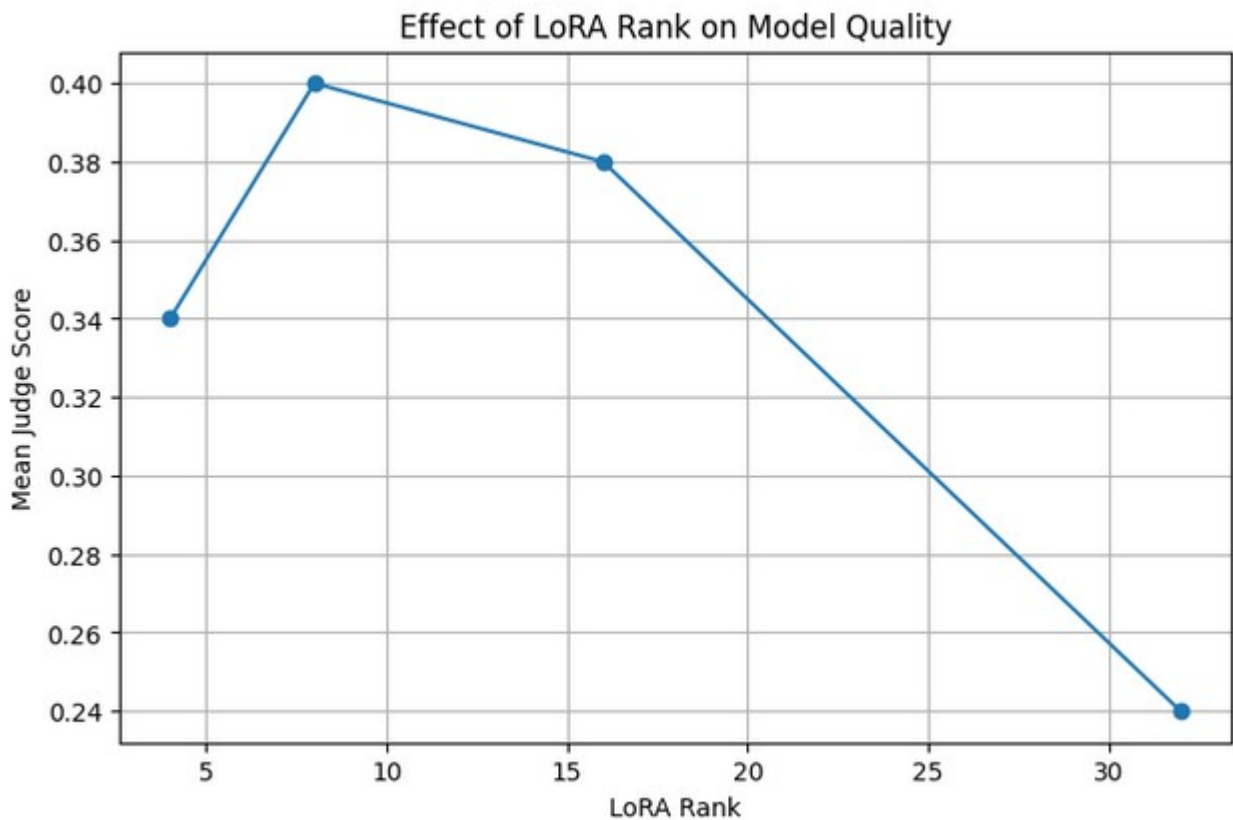
print("\nBEST CONFIGURATION")
print(best_run)

Paris, the capital of France, is.
step 540 loss: 1.9094
Capital of France, Paris, is.
step 550 loss: 2.4516
Capital of France, Paris is.

```

В результаті маємо порівняльну таблицю, графік та висновки

	LoRA_rank	Trainable_params	Generated_score_mean	Generated_score_std	StyleGain
0	4	221184	0.34	0.320000	34.0
1	8	442368	0.40	0.228035	40.0
2	16	884736	0.38	0.248193	38.0
3	32	1769472	0.24	0.162481	24.0



```
BEST CONFIGURATION
LoRA_rank          8.000000
Trainable_params   442368.000000
Generated_score_mean 0.400000
Generated_score_std 0.228035
StyleGain          40.000000
```

Зі збільшенням значення LoRA Rank кількість тренованих параметрів помірно зменшувалася. Проте залежність між значенням рангу та якістю моделі виявилася нелінійною.

При переході від rank = 4 до rank = 8 спостерігалось незначне зростання якості генерації (з 0.34 до 0.40). Для rank = 16 якість трохи погіршилася: середній Score знизився до 0.38, а StyleGain — до 38%. Це свідчить про те, що

збільшення кількості параметрів саме по собі не гарантує покращення результатів.

Найгірші результати були отримані для $\text{rank} = 32$. Дана конфігурація продемонструвала мінімальний Generated Score (0.24). Це означає, що модель найгірше відтворювала особливості стилю Yoda саме при цьому значенні рангу.

Побудований графік підтверджує отримані результати. На ньому видно зростання якості для $\text{rank} = 8$ та $\text{rank} = 16$ і суттєве погіршення для $\text{rank} = 32$. Найвища точка графіка відповідає конфігурації $\text{rank} = 8$.

Отже, в ході дослідження було встановлено, що параметр LoRA Rank суттєво впливає на якість донавчання моделі. А найкращою серед протестованих конфігурацій виявилася LoRA Rank = 8.

2.9 Порівняння результатів до та після Fine-Tuning

Для демонстрації впливу Fine-Tuning обремо п'ять тестових запитань загального характеру. Спочатку відповіді на ці запитання згенеруємо базовою моделлю. Після цього донавчимо модель на датасеті відповідей у стилі Yoda та повторно задамо ті самі запитання.

Для кожного прикладу виконаємо оцінювання за допомогою методу LLM-as-a-Judge. Суддя оцінить ступінь відповідності відповіді стилю Yoda за шкалою від 0 до 1.

Програмний код:

```
[35] ✓ 4m
import pandas as pd
import torch

# -----
# ПИТАННЯ ДЛЯ ПОРІВНЯННЯ
# -----

test_questions = [
    "What is the capital of France?",
    "Who invented the telephone?",
    "What is machine learning?",
    "How do I make a cup of tea?",
    "What is the largest ocean on Earth?"
]

# -----
# ВІДПОВІДІ ДО FINE-TUNING
# -----

before_samples = []

print("Generating answers BEFORE fine-tuning...")

for question in test_questions:
    with torch.no_grad():
        answer = chat(
            question,
            only_answer=True,
            max_new_tokens=48,
            temperature=0.3
        )["answer"]

        before_samples.append(answer)

# -----
# FINE-TUNING
# -----

model = train(
    model,
    train_loader,
    tokenizer,
    max_steps=1600,
    learning_rate=5e-5,
    run_name="yoda"
)
```

```
after_samples = []

print("Generating answers AFTER fine-tuning...")

for question in test_questions:
    with torch.no_grad():
        answer = chat(
            question,
            only_answer=True,
            max_new_tokens=48,
            temperature=0.3
        )["answer"]

        after_samples.append(answer)

# -----
# JUDGE EVALUATION
# -----

before_scores = evaluate_samples(
    before_samples,
    "before_finetuning"
)

after_scores = evaluate_samples(
    after_samples,
    "after_finetuning"
)

# -----
# TABLE
# -----

comparison_df = pd.DataFrame({
    "Prompt": test_questions,
    "Before Fine-tuning": before_samples,
    "Score Before": before_scores,
    "After Fine-tuning": after_samples,
    "Score After": after_scores
})

comparison_df["Improvement"] = (
    comparison_df["Score After"]
    - comparison_df["Score Before"]
)

display(comparison_df)
```


Після завершення оцінювання сформувалася порівняльна таблиця

	Prompt	Before Fine-tuning	Score Before	After Fine-tuning	Score After	Improvement
0	What is the capital of France?	The capital of France is Paris. This city is n...	0.0	Paris, the capital of France, is.	0.2	0.2
1	Who invented the telephone?	The invention of the telephone is credited to ...	0.0	Invented, the telephone was. By Alexander Grah...	0.5	0.5
2	What is machine learning?	Machine learning is a subset of artificial int...	0.0	Machine learning, a type of artificial intelli...	0.3	0.3
3	How do I make a cup of tea?	Making a cup of tea is a simple process! Here'...	0.0	To make a cup of tea, you must boil water. Add...	0.0	0.0
4	What is the largest ocean on Earth?	The largest ocean on Earth is the Pacific Ocea...	0.0	The Pacific, it is. The largest ocean on Earth...	0.3	0.3

Результати демонструють, що Fine-Tuning успішно адаптував модель до генерації відповідей у стилі Yoda. У чотирьох із п'яти тестових прикладів спостерігається покращення оцінки судді. Після донавчання модель почала використовувати характерний порядок слів і мовні конструкції, властиві персонажу Yoda.

Водночас результати показують, що адаптація відбулася не однаково для всіх типів запитів. Для деяких питань стилістичні зміни були виражені сильніше, ніж для інших. Незважаючи на це, загальна тенденція свідчить про успішне перенесення цільового стилю на відповіді моделі та підтверджує ефективність застосованого Fine-Tuning.

3. Висновки: в ході роботи було досліджено процес донавчання (fine-tuning) мовної моделі за допомогою LoRA та оцінено якість отриманих результатів. Було проаналізовано вплив параметрів навчання на якість моделі та підтверджено ефективність Fine-Tuning для адаптації моделі до стилю Yoda.

4. Посилання

Файл в репозиторії гіт:

https://github.com/NureTymoshchenkoVladyslava/GNMS_Labs/tree/main/Lb1